Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – макроопределения»

Выполнил:
Студент 4-го курса
Группы АС-66
Куган Н. Л.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: изучение приёмов разработки макроопределений и использования их в программах.

Задание

1. Изучить состав и средства задания макроопределений.

2. Написать макроопределение, реализующее функцию заданного преподавателем варианта работы.

3. Выполнить проверку работоспособности разработанного макроопределения.

Вариант 9

Задана строка слов. Словом считается последовательность символов, разделённая пробелами. Из исходной строки удалить слова, начинающиеся с заданного символа.

Реализация

Программа разработана в MS Visual Studio (консольное приложение C++, платформа x86) и состоит из двух файлов. В файле lab3_main.cpp выполнены ввод и вывод: функция ReadText читает текст до точки в буфер из 100 символов и определяет, что ввод не является текстом (пустой ввод, Enter до точки, более 100 символов), затем вводится искомый символ. Консоль переключена в кодировку Windows-1251. В файле lab3_asm.asm (MASM, x86) записано макроопределение REMOVE_WORDS и процедура-обёртка RemoveWordsAsm, в теле которой макрос разворачивается. Процедура вызывается из C++ по соглашению cdecl и объявлена как extern "C" int RemoveWordsAsm(char* text, int len, char target); она возвращает новую длину текста.

Макроопределение состоит из заголовка (имя REMOVE_WORDS, псевдооператор MACRO и четыре формальных параметра: strAddr – адрес строки, strLen – длина, targetChar – искомый символ, result – регистр для новой длины), тела и концевика ENDM. Метки внутри тела объявлены директивой LOCAL, поэтому при каждом макровызове ассемблер создаёт собственные метки и повторное использование макроса не приводит к конфликту имён. Фактическими параметрами могут быть регистры, ячейки памяти или

константы; в программе макрокоманда вызывается с операндами [ebp+8], [ebp+12], byte ptr [ebp+16] и регистром eax.

Алгоритм: ESI – указатель чтения, EDI – указатель записи, EBX – начало строки, ECX – число необработанных символов, DL – искомый символ, AL – текущий символ. Пробел (код 20h) переписывается без изменений. Любой другой символ является первым символом слова: если он равен искомому, слово пропускается (ESI перемещается до ближайшего пробела или конца строки, EDI не изменяется), иначе слово копируется на позицию EDI. Новая длина строки вычисляется как EDI − EBX и помещается в result (регистр EAX). Текст преобразуется на месте, дополнительная память и глобальные переменные не используются.

Принятые допущения: удаляется только само слово, пробелы между словами не изменяются (после удаления слов могут остаться подряд идущие пробелы); символ сравнивается с первым символом слова с учётом регистра (П и п – разные символы); если искомый символ – пробел, ни одно слово не удаляется.

Текст программы

lab3_main.cpp

```
// lab3_main.cpp
// Лабораторная работа №3, вариант 9. Куган Н. Л., АС-66. Платформа: x86.
// Ввод строки слов и символа, вызов макроопределения, вывод результата.
//
// Задание: задана строка слов (слово - последовательность символов между
// пробелами). Из строки удалить слова, начинающиеся с заданного символа.
// Алгоритм реализован в виде макроопределения REMOVE_WORDS (см. lab3_asm.asm).
// Пробелы между словами не изменяются, регистр символа учитывается.
//
// Строки для проверки (вводить в консоль вместе с точкой; затем символ):
//   apple banana avocado cherry.   символ a  -> " banana  cherry"
//   Привет мир при пока.           символ п  -> "Привет мир  " (удалены "при" и "пока")
//   Привет мир при пока.           символ П  -> " мир при пока"
//   one two three.                 символ x  -> "one two three" (без изменений)
//   abc abd abe.                   символ a  -> "  " (удалены все слова)
//   a1 b2 a3 c4.                   символ a  -> " b2  c4"
//   hello.                         символ h  -> "" (пустая строка)
//   .                              -> ошибка: пустой ввод
//   ТЕКСТ БЕЗ ТОЧКИ                -> ошибка: нет точки (Enter до точки)

#include <cstdio>
#include <windows.h>

#ifndef _M_IX86
#error Assembly module is built with ml.exe (x86 only): choose platform "x86" (Win32).
#endif

// реализация в lab3_asm.asm: обёртка над макросом REMOVE_WORDS, возвращает новую длину текста
```

```
extern "C" int RemoveWordsAsm(char* text, int len, char target);

const int MAX_LEN = 100;

int ReadText(char* buf, int& len)
{
    len = 0;
    for (;;)
    {
        int c = std::getchar();
        if (c == '.')
            break;
        if (c == '\n' || c == EOF)
            return 2;
        if (len >= MAX_LEN)
            return 2;
        buf[len++] = (char)c;
    }
    return (len == 0) ? 1 : 0;
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    char text[MAX_LEN + 1] = { 0 };
    int len = 0;

    std::printf("=========================================================\n");
    std::printf(" Lab 3, variant 9. Macro definitions\n");
    std::printf(" Task: the text is a string of words (separated by\n");
    std::printf("       spaces). Delete all words that begin with\n");
    std::printf("       the given character.\n");
    std::printf("=========================================================\n\n");
    std::printf("Enter text (max 100 chars, end with '.'):\n> ");

    int status = ReadText(text, len);
    if (status == 1)
    {
        std::printf("\nError: empty input, this is not a text!\n");
        return 0;
    }
    if (status == 2)
    {
        std::printf("\nError: no '.' within 100 characters, this is not a text!\n");
        return 0;
    }

    // после точки в потоке остаётся остаток строки (Enter) - убираем его,
    // иначе следующий getchar() прочитает его вместо искомого символа
    int c;
    while ((c = std::getchar()) != '\n' && c != EOF)
    {
    }

    std::printf("Enter the character that words must begin with:\n> ");
    int t = std::getchar();
    if (t == '\n' || t == EOF)
    {
        std::printf("\nError: character is not entered!\n");
        return 0;
    }
    char target = (char)t;

    std::printf("\nOriginal text   : %.*s\n", len, text);
    std::printf("Character       : '%c'\n", target);

    int newLen = RemoveWordsAsm(text, len, target);
```

```
    std::printf("Result length   : %d\n", newLen);
    std::printf("Transformed text: [%.*s]\n", newLen, text);
    return 0;
}
```

lab3_asm.asm

```
; lab3_asm.asm
; Лабораторная работа №3 (ассемблер, макроопределения). Вариант 9.
; Куган Н. Л., АС-66.
;
; Задание:
;     задана строка слов; словом считается последовательность символов,
;     разделённая пробелами. Из исходной строки удалить слова,
;     начинающиеся с заданного символа.
;
; Пробелы между словами не изменяются (удаляется само слово),
; символ сравнивается с первым символом слова с учётом регистра.
;
; extern "C" int RemoveWordsAsm(char* text, int len, char target);
;
; Платформа проекта - x86 (Win32), сборка через ml.exe.


.386
.MODEL FLAT, C
.CODE


; ---------------------------------------------------------------------
; Макроопределение REMOVE_WORDS
; Параметры:
;   strAddr    - адрес начала строки (регистр или память)
;   strLen     - длина строки (регистр, память или константа)
;   targetChar - искомый символ (регистр или память, один байт)
;   result     - регистр-приёмник новой длины строки
;
; Строка преобразуется на месте. Макрос использует регистры
; esi (чтение), edi (запись), ebx (начало строки), ecx (счётчик),
; edx (dl - искомый символ) и al (текущий символ), поэтому result
; должен быть регистром, отличным от esi, edi, ebx, ecx и edx.
; Вызывающая процедура сама сохраняет esi, edi и ebx.
; ---------------------------------------------------------------------
REMOVE_WORDS MACRO strAddr, strLen, targetChar, result
    LOCAL nextItem, wordStart, copyWord, skipWord, scanDone

    mov     esi, strAddr        ; esi = откуда читаем
    mov     edi, esi            ; edi = куда пишем
    mov     ebx, esi            ; ebx = начало строки (для вычисления длины)
    mov     ecx, strLen         ; ecx = сколько символов ещё не обработано
    mov     dl, targetChar      ; dl = искомый символ

nextItem:
    test    ecx, ecx
    jz      scanDone            ; строка закончилась
    mov     al, [esi]
    cmp     al, 20h             ; пробел?
    jne     wordStart
    mov     [edi], al           ; пробел переписываем без изменений
    inc     esi
    inc     edi
    dec     ecx
    jmp     nextItem

wordStart:                      ; al - первый символ слова
    cmp     al, dl
    je      skipWord            ; слово начинается с искомого символа - удаляем

copyWord:                       ; копируем слово до пробела или конца строки
    test    ecx, ecx
    jz      scanDone
```

```
    mov     al, [esi]
    cmp     al, 20h
    je      nextItem
    mov     [edi], al
    inc     esi
    inc     edi
    dec     ecx
    jmp     copyWord

skipWord:                       ; пропускаем слово до пробела или конца строки
    test    ecx, ecx
    jz      scanDone
    cmp     byte ptr [esi], 20h
    je      nextItem
    inc     esi
    dec     ecx
    jmp     skipWord

scanDone:
    mov     result, edi
    sub     result, ebx         ; result = edi - начало = новая длина строки
ENDM



; ------------------------------------------------------------------
; int RemoveWordsAsm(char* text, int len, char target)
; Процедура-обёртка: принимает параметры по cdecl и вызывает макрос
; REMOVE_WORDS, который разворачивается прямо здесь, в теле процедуры.
; Возвращает в eax новую длину текста.
; ------------------------------------------------------------------
PUBLIC RemoveWordsAsm
RemoveWordsAsm PROC
    push    ebp
    mov     ebp, esp
    push    esi                 ; esi, edi, ebx - callee-saved по cdecl
    push    edi
    push    ebx

    REMOVE_WORDS [ebp+8], [ebp+12], byte ptr [ebp+16], eax

    pop     ebx
    pop     edi
    pop     esi
    pop     ebp
    ret
RemoveWordsAsm ENDP

END
```

Результаты работы

Программа собрана в Visual Studio (x86) и запущена; ниже приведены контрольные примеры. Результат показан так, как его выводит программа: в квадратных скобках, чтобы были видны оставшиеся пробелы.

№	Ввод (текст / символ)	Результат	Проверяет
1	apple banana avocado cherry. / a	[banana cherry], длина 15	удаление слов на «a»
2	Привет мир при пока. / п	[Привет мир], длина 12	русские буквы, учёт регистра (П ≠ п)
3	one two three. / x	[one two three], длина 13	нет подходящих слов
4	abc abd abe. / a	[ ], длина 2	удалены все слова
5	a1 b2 a3 c4. / a	[b2 c4], длина 7	слова с цифрами
6	hello. / h	[], длина 0	единственное слово
7	«.»	ошибка: пустой ввод	не текст
8	ТЕКСТ БЕЗ ТОЧКИ + Enter	ошибка: нет точки	не текст

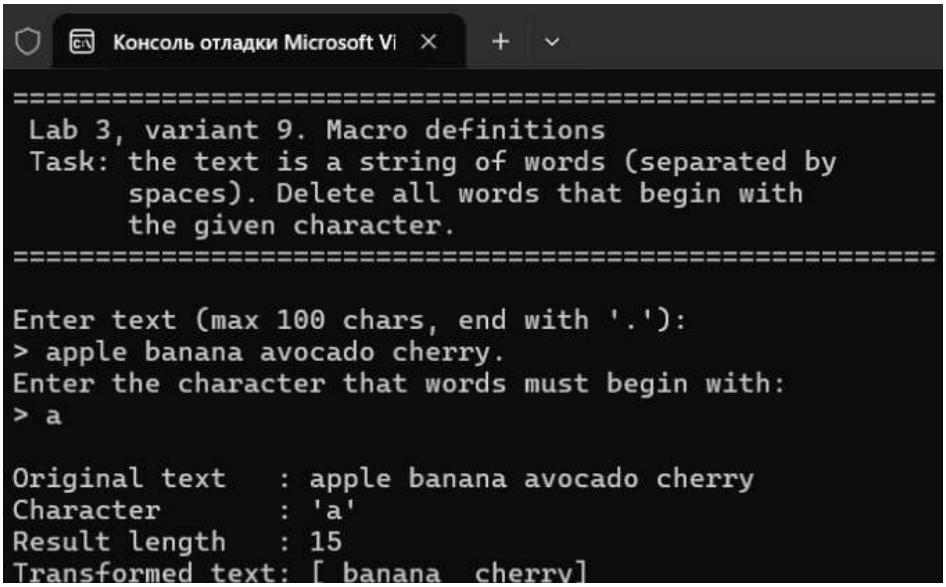


Рисунок 1 – Удаление слов, начинающихся с «а»

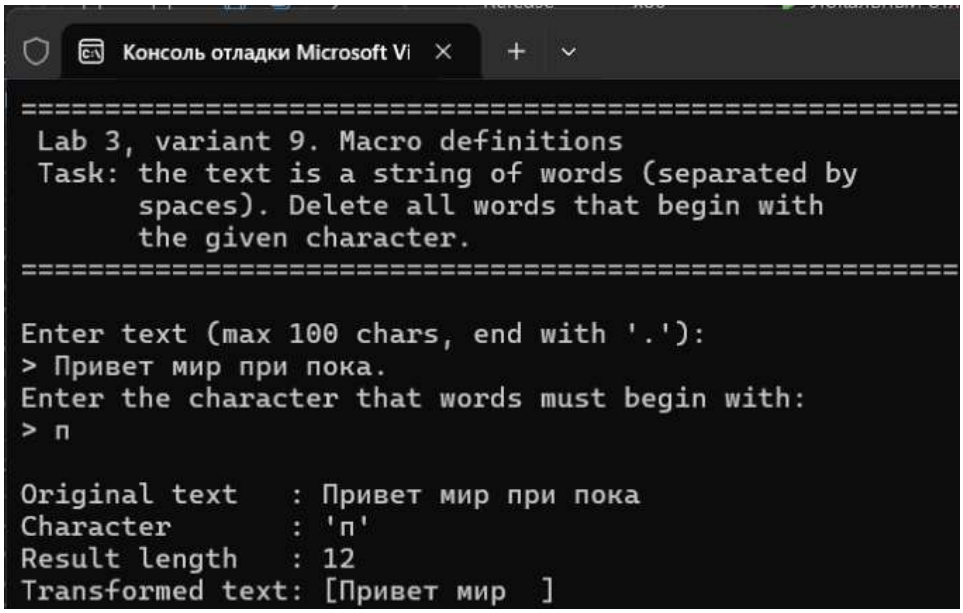


Рисунок 2 – Русские буквы, регистр учитывается (П ≠ п)

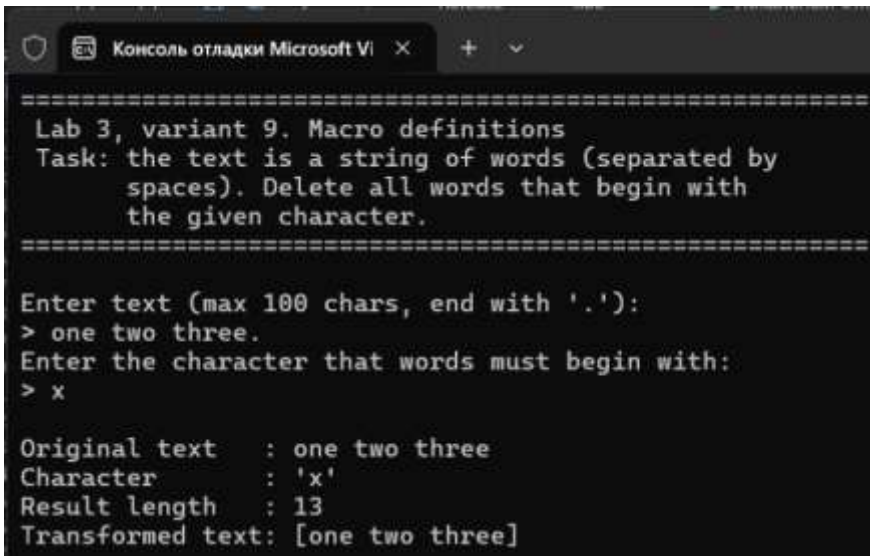


Рисунок 3 – Нет слов, начинающихся с заданного символа

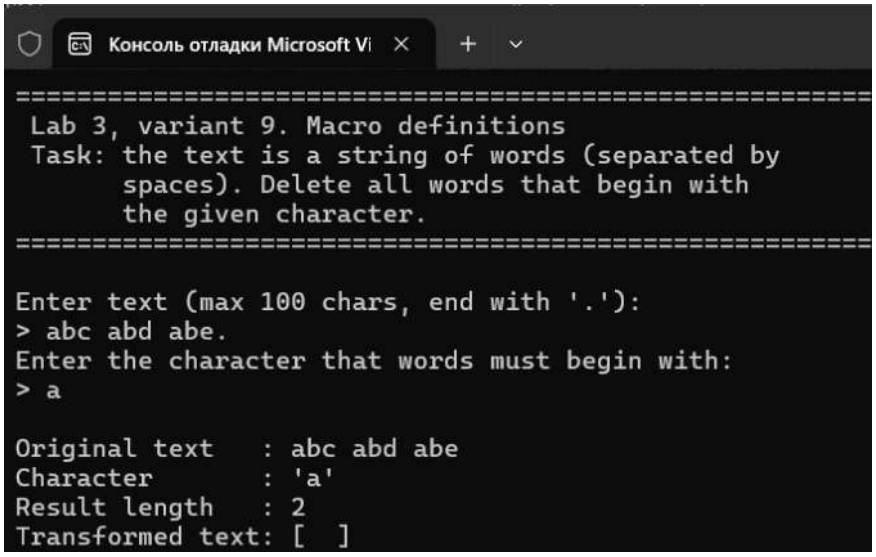


Рисунок 4 – Удалены все слова

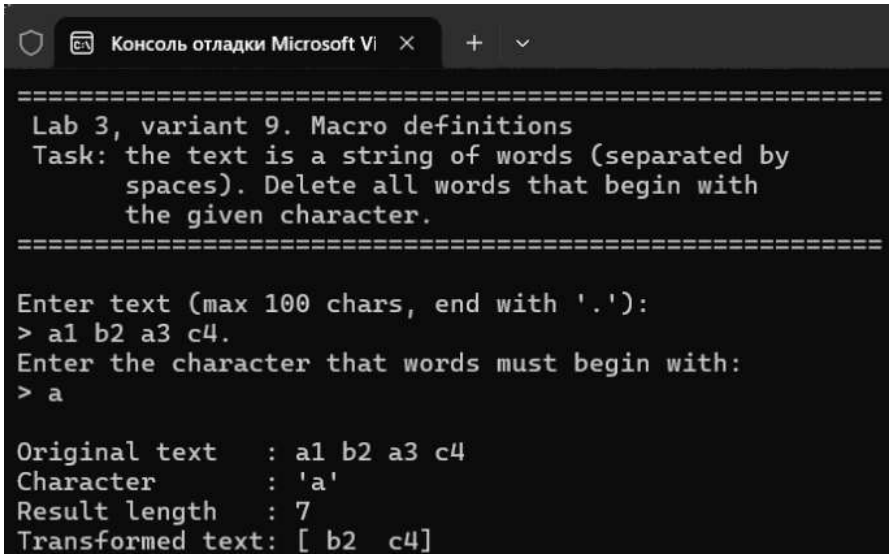


Рисунок 5 – Слова с цифрами

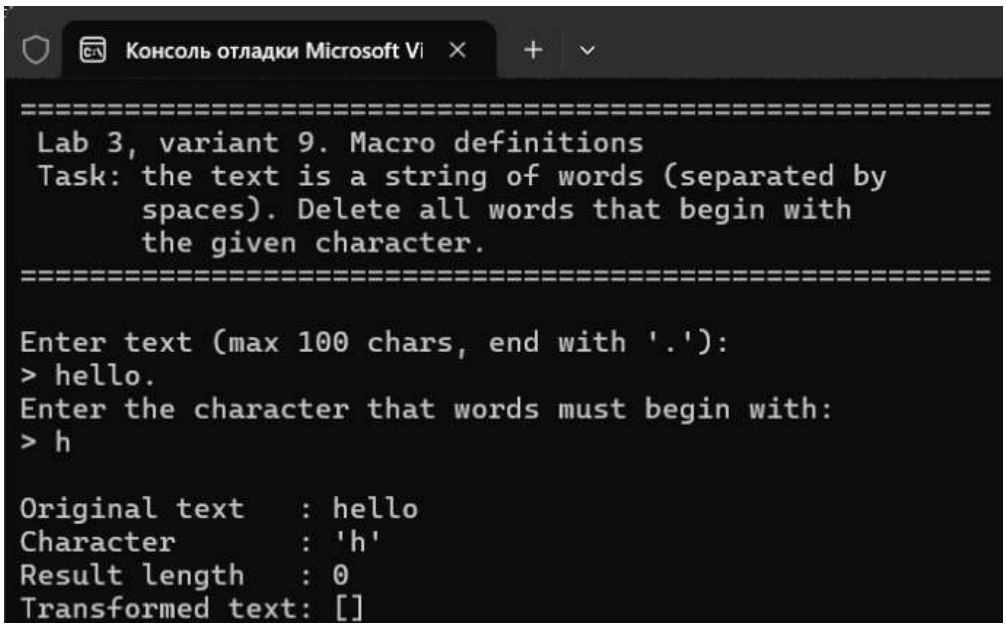


Рисунок 6 – Единственное слово удалено, результат – пустая строка

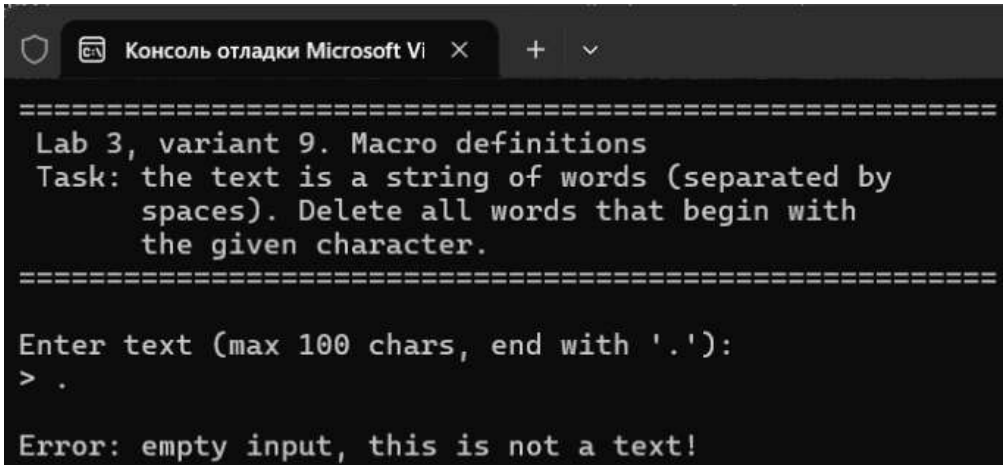


Рисунок 7 – Пустой ввод – не текст

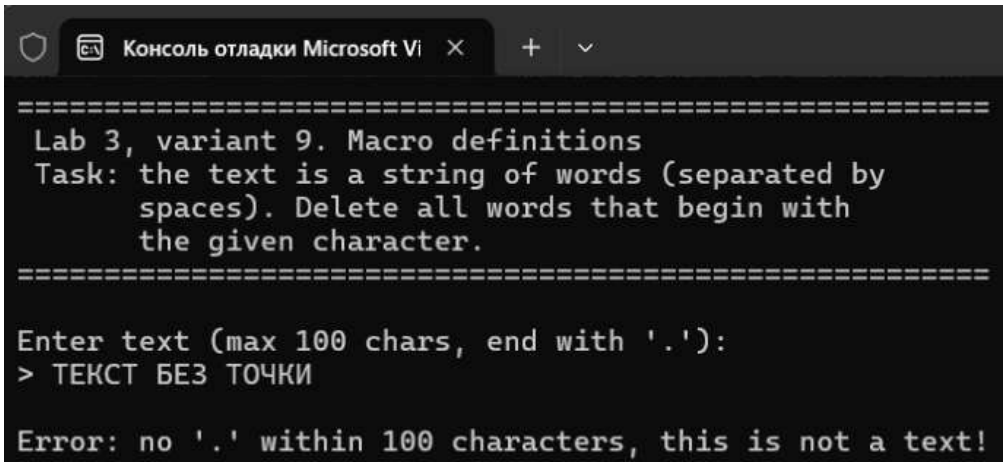


Рисунок 8 – Нет точки – не текст